

Server-Side WebAssembly

in Practice

Skaara Poncin · Storm van Loon · Gabriel Uwaila

Learning goals

- Understand what Server-Side Wasm is
- Explain how Wasm + WASI work together
- Compare modules vs components
- See practical use cases: HTTP, AI, edge/cloud
- Know current limitations and when to use Wasm

■ What is Server-Side WebAssembly?

- WebAssembly (`.wasm`) is a portable binary format
- Usually known from browsers, but also runs on servers
- A runtime (e.g. Wasmtime) executes Wasm in a sandbox
- Goal: safe, fast, portable execution across environments

Why should students care?

- Performance close to native for many workloads
- Strong isolation for untrusted or third-party code
- Good for polyglot systems (Rust, Go, C/C++, etc.)
- Useful in modern systems: edge, serverless, plugins, AI helpers

How execution works (high level)

- Compile source code to `.wasm`
- Runtime loads + validates the module
- Runtime compiles/interprets for the host CPU
- Code runs inside a sandbox with controlled access
- Host app and Wasm app communicate through imports/exports

WASI (why it matters)

- Wasm alone has no direct filesystem/network access
- WASI provides standardized system interfaces
- Examples: HTTP handling, clocks/timers, I/O capabilities
- Key idea: explicit permissions, not full machine access

Security model

- Capabilities must be granted explicitly
- Sandbox limits impact of bugs and malicious code
- Components can improve isolation further (shared-nothing)
- Trade-off: `.wasm` is binary, so debugging often uses `.wat`

Runtime ecosystem (examples)

- General/server: Wasmtime, Wasmer, WasmEdge
- JavaScript world: Node and Deno support Wasm too
- IoT-focused: WAMR, wasm3
- Choice depends on target platform + feature support

Runtimes	js / web	java	IOT devices	server / general	education
node.js	✓				
deno	✓				
GraalWasm		✓			
Chicory		✓			
Wasm3			✓		
WAMR			✓		
Wasmtime				✓	
Wasmer				✓	
WasmEdge				✓	
Wazero				✓	
Wizard					✓

Modules vs Components

- Module: basic unit, simpler, widely supported
- Component: higher-level packaging with better interoperability
- Components are designed for composition and safer boundaries
- In practice: support is improving, but not equal everywhere

WIT in one slide

- WIT = WebAssembly Interface Types
- Describes function contracts and data types across languages
- Helps avoid custom glue code for every language pair
- Enables more reliable cross-language integration

Example 1: HTTP service with Wasm

- We implemented `wasi:http/incoming-handler`
- Same service idea across Rust, TinyGo, and C++
- Shows portability of interface-based development
- Main lesson for students: same contract, different language ergonomics

Example 2: AI inference with Wasm

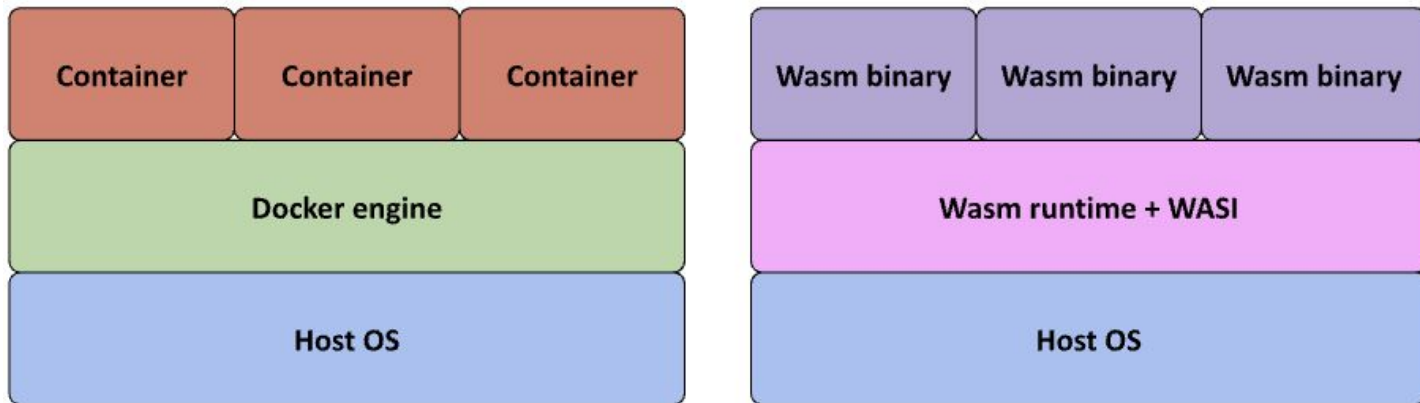
- Image in -> vector embedding out (ONNX-based flow)
- Inference logic in Rust compiled to Wasm
- Good fit for sandboxed edge/serverless execution
- Important practical point: warm/cached execution matters

Where Server-Side Wasm fits best today

- Edge/CDN compute close to users
- Serverless functions with fast cold starts
- Plugin systems with untrusted extensions
- Sandboxed compute near data (e.g., database UDF scenarios)

Cloud and deployment landscape

- Hosts/platforms: Spin and WasmCloud are common entry points
- Kubernetes integration exists but can require extra setup
- Not a full replacement for containers in every case
- Best viewed as an additional tool in your architecture toolbox



Current limitations (important for engineering decisions)

- Tooling maturity varies by language/runtime
- Documentation quality is inconsistent across projects
- New WASI/component features are not uniformly implemented
- Debugging and observability are still less mature than mainstream stacks

Takeaway for students

- Server-Side Wasm is practical, not just hype
- Think of it as: portable sandboxed compute
- Use it where isolation, startup speed, or portability matter most
- Keep expectations realistic: evaluate runtime/tooling support first